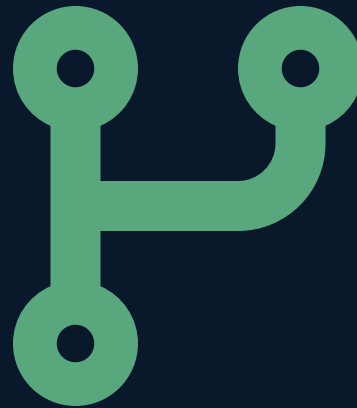


PORTADA

Técnicas de Programación

Unidad 0 — Paradigma Imperativo: Estructuras de Control

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

**El mismo bloque de código.
Ejecutado una vez... o un millón.**



CONCEPTO

Decidir con `if / else`

```
if (nota >= 3.0) {  
    System.out.println("Aprobado");  
} else {  
    System.out.println("Reprobado");  
}
```

El bloque `if` se ejecuta solo si la condición (una expresión `boolean` , como las de la sesión anterior) es `true` ; el bloque `else` , solo si es `false` .

CONCEPTO

Más de dos caminos: else if y switch

```
if (nota >= 4.5) categoria = "Excelente";  
else if (nota >= 3.5) categoria = "Bueno";  
else if (nota >= 3.0) categoria = "Aprobado";  
else categoria = "Reprobado";
```

Cuando hay varias condiciones mutuamente excluyentes sobre una misma variable, `switch` suele ser más legible que una cadena larga de `else if` :

```
switch (dia) {  
    case 1 -> System.out.println("Lunes");  
    case 2 -> System.out.println("Martes");  
    default -> System.out.println("Otro día");  
}
```

CONCEPTO

El ciclo for

Se usa cuando **se conoce (o se puede calcular) el número de repeticiones** antes de entrar al ciclo. Tiene tres partes separadas por punto y coma:

```
for (int i = 0; i < 5; i++) {  
    System.out.println("Vuelta número " + i);  
}
```

Parte	Qué hace
int i = 0	Inicialización — se ejecuta una sola vez, al entrar
i < 5	Condición — se revisa antes de cada vuelta
i++	Incremento — se ejecuta al final de cada vuelta



CONCEPTO

El ciclo while

Se usa cuando **no se sabe de antemano cuántas veces se va a repetir** — solo se sabe la condición de parada.

```
int intentos = 0;
while (intentos < 3) {
    System.out.println("Intento " + intentos);
    intentos++;
}
```

La condición se evalúa **antes** de cada vuelta — si es `false` desde el principio, el bloque no se ejecuta ni una sola vez.

CONTRA EJEMPLO

La diferencia con do-while

```
int clave;  
do {  
    clave = leerClave();  
} while (clave != claveCorrecta);
```

En `do-while` el bloque se ejecuta **primero** y la condición se evalúa **después** — por eso el bloque siempre corre al menos una vez, incluso si la condición ya era falsa. Es el ciclo natural para "pedir algo hasta que sea válido".



CONCEPTO

Recorrido simplificado: for-each

```
double[] notas = {3.5, 4.0, 2.8, 4.5};  
for (double nota : notas) {  
    System.out.println(nota);  
}
```

`for-each` recorre cada elemento de una colección sin necesitar un índice ni controlar manualmente la condición de parada — a cambio, no permite conocer la posición del elemento ni modificar la colección mientras se recorre.



CONCEPTO

Acumuladores y contadores

Dos patrones que aparecen en casi todo ciclo:

- **Contador** — una variable que suma o resta una cantidad fija en cada vuelta (`i++` , `i--`), típicamente para controlar cuántas veces se repite.
- **Acumulador** — una variable que va guardando un resultado que crece con cada vuelta (`suma += nota;`), típicamente para totalizar algo al final del ciclo.

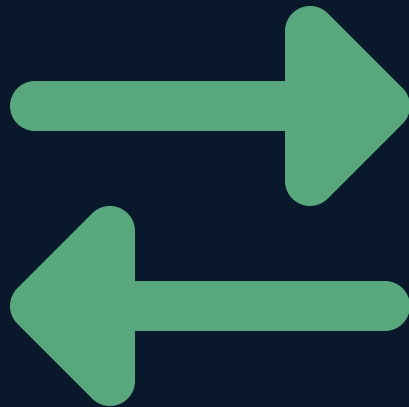
```
double suma = 0;
for (int i = 0; i < notas.length; i++) {
    suma += notas[i];
}
```

CONCEPTO

Convertir entre tipos: casting

Tipo de conversión	Cuándo ocurre	Ejemplo
Widening (ampliación)	Automática, de un tipo pequeño a uno más grande	<code>int a = 10; double b = a;</code>
Narrowing (reducción)	Manual, con el tipo entre paréntesis; puede perder información	<code>double c = 9.7; int d = (int) c; // d vale 9</code>

El narrowing trunca, no redondea — `(int) 9.7` da 9, no 10 .



INTERACCIÓN

Sumar con un ciclo

Escriban un programa que declare un arreglo con 5 números y use un ciclo `for` para calcular e imprimir la suma total y el promedio.

10 minutos · resuelvan individualmente, luego comparen el resultado

SÍNTESIS

Lo que hay que llevarse de hoy

1. `if / else` y `switch` controlan qué bloque de código se ejecuta según una condición.
2. `for` se usa cuando se conoce el número de repeticiones; `while` y `do-while`, cuando depende de una condición que puede cambiar.
3. `do-while` garantiza al menos una ejecución porque evalúa la condición al final.
4. `for-each` simplifica el recorrido de colecciones a cambio de no exponer el índice.
5. El casting narrowing trunca decimales al reducir de `double` a `int`.



CIERRE

Antes de la próxima sesión

- Practicar un ciclo `for` que recorra un arreglo de al menos 5 posiciones.
- Traer resuelto (en borrador) el ejercicio de acumulador de hoy — se retoma en la próxima clase.

La próxima sesión cierra la Unidad 0 con vectores: arreglos, matrices y ArrayList — y el taller grupal de cierre.

